Java Silver（SE 17・1Z0-825-JPN）　2 Javaの基本データ型と文字列の操作　#02

変数とデータ型

8 つの基本データ型と初期値、リテラルと識別子、ローカル変数の初期化とスコープ、拡大・縮小変換とキャスト、ラッパークラス、var（ローカル変数型推論）

各ページの下に、根拠にした Oracle の資料（JLS・API）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 2（変数・var）

1 基本データ型
8 つの型の大きさと範囲、初期値、オーバーフロー

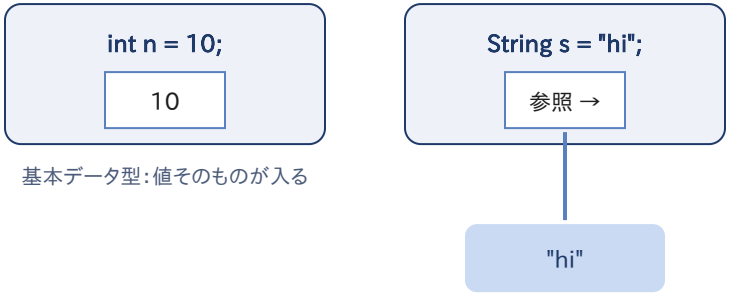
2 リテラルと識別子
L・F・0x・0・0b・アンダースコア、名前の決まり、予約語と文脈キーワード

3 宣言・初期化・スコープ
ローカル変数は初期値なし、ブロックの範囲、同じ名前の宣言

4 型変換とラッパークラス
拡大・縮小、キャスト、定数の特別ルール、+=、ボクシング

5 var
初期化の式から型が 1 つに決まる。使えない書き方

変数は「型の付いた箱」（Claude が作成）



芯：型はコンパイルのときに決まり、入る値の範囲と変換のしかたを決める。var でも同じ。

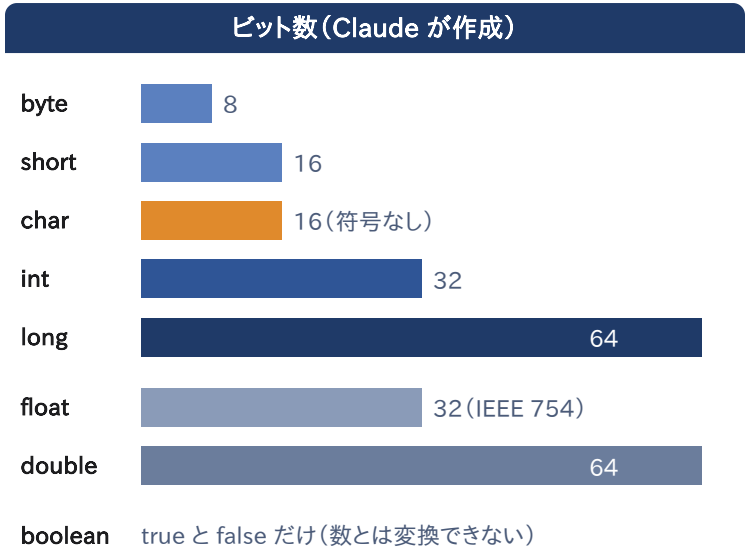
出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト（変数の宣言、初期化とスコープ／ローカル変数型推論の使用）／JLS（Java SE 17）4.1（There are two kinds of types in the Java programming language: primitive types and reference types）

第1章

基本データ型

8 つの型。大きさと、範囲をこえたときの動きを覚える。

8 つの型の大きさと範囲



型	範囲	初期値
byte	-128 ～ 127	0
short	-32768 ～ 32767	0
char	0 ～ 65535（符号なし）	'\u0000'
int	約 ±21 億（2^{31}）	0
long	約 ±922 京（2^{63}）	0L
float・double	小数	0.0
boolean	true・false	false
参照型	参照か null	null

まちがえやすい：char と short はどちらも 16 ビットだが、**char は符号なし**（負の数なし）、short は符号あり。だから char ⇔ short はキャストが要る。

出典：JLS 4.2（byte, short, int, and long, whose values are 8-bit, 16-bit, 32-bit and 64-bit signed two's-complement integers, respectively, and char, whose values are 16-bit unsigned integers／float … 32-bit IEEE 754 … double … 64-bit）／4.2.1（For byte, from -128 to 127, inclusive）／4.12.5（default value）

初期値があるのはフィールドと配列だけ

フィールドと配列（Claude が作成）

```
public class Main {
  int x;            // フィールド
  String s;
  public static void main(String[] args) {
    Main m = new Main();
    int[] a = new int[2];
    System.out.println(m.x + " " + m.s + " " + a[1]);
  }
}
```

```
0 null 0
```

ローカル変数

```
int x;
if (args.length > 0) x = 1;
System.out.println(x);
```

```
error: variable x might not have been initialized
```

変数	使う前の値
フィールド	型の初期値（0・false・null など）
配列の要素	型の初期値
メソッドの引数	呼んだ側が渡した値
ローカル変数	**なし**。使う前に代入が必要

確実な代入：コンパイラは「どの道すじでも値が入るか」を調べる。if だけで代入すると、条件が false の道で値がないのでエラー。**if と else の両方**で代入すれば OK。

宣言だけならエラーにならない：`int x;` だけでは何も起きない。エラーは**読んだ所**で出る。

出典：JLS 4.12.5 Initial Values of Variables（Each class variable, instance variable, or array component is initialized with a default value when it is created／A local variable … must be explicitly given a value before it is used … using the rules for definite assignment）／JLS 16 Definite Assignment／結果は JDK 17 で確かめたもの

範囲をこえると回りこむ（オーバーフロー）

int の数直線は輪になっている（Claude が作成）




```
int x = 2147483647;
long y = x + 1;     // int で計算してから long へ
long z = x + 1L;    // long で計算
System.out.println(y + " " + z);
```

```
-2147483648 2147483648
```

エラーにも例外にもならない。整数の計算が範囲をこえると、静かに反対の端へ回りこむ。

代入先は関係ない：計算の型は右辺の式で決まる。`x + 1` は int どうしなので int で回りこみ、そのあと long に広がる。片方を `1L` にすれば long で計算される。

出典：JLS 5.6 Numeric Contexts（if any expression is of type long, then the promoted type is long … Otherwise … the promoted type is int）／JLS 4.2.2（The integer operators do not indicate overflow or underflow in any way）／結果は JDK 17 で確かめたもの

第2章

リテラルと識別子

書いた値の型は、書き方で決まる。名前には決まりがある。

整数リテラルと小数リテラル

書き方	型	値・注意
`100`	int	何も付けなければ int
`2147483648L`	long	L(l)で long。L なしは integer number too large
`0x1F`	int	16 進 → 31
`017`	int	0 始まりは 8 進 → 15。`09` はエラー
`0b101`	int	2 進 → 5
`1.5`・`1.5D`	double	小数は何も付けなければ double
`1.5F`	float	F(f)で float
`'A'`・`'A'`	char	1 文字。`"A"` は String

JDK 17 の結果

```
long a = 2147483648;  // integer number too large
float f = 1.5;        // lossy conversion from double
float g = 10;         // OK (int → float) → 10.0
long l = 1.0;         // lossy conversion from double
boolean b = 0;        // int cannot be converted to boolean
char c = 65;          // OK → 'A'
```

代入先ではなく、リテラル自体の型を見る。`long a = 2147483648;` は、右辺がまず int として読まれるのでエラー。

8 進の計算:017 = 1×8+7 = 15。**16 進:**0x1F = 1×16+15 = 31。**2 進:**0b101 = 4+1 = 5。

出典:JLS 3.10.1 Integer Literals(An integer literal is of type long if it is suffixed with an ASCII letter L or l; otherwise it is of type int／An octal numeral consists of an ASCII digit 0 followed by one or more of the ASCII digits 0 through 7)／3.10.2(of type float if it is suffixed with an ASCII letter F or f; otherwise its type is double)／結果は JDK 17 で確かめたもの

アンダースコアの置き場所

書き方	結果	理由
`1_000_000`	OK	数字と数字の間
`1__0`	OK	続けて 2 つも可（値は 10）
`0_17`	OK	8 進の 0 も数字なので、その後ろは可（15）
`1_000.5_0`	OK	小数部分の数字の間も可
`100_`	エラー	最後の数字のあと
`0x_1F`	エラー	0x・0b の直後
`1._5・1_.5`	エラー	小数点の前後
`10_L・1.5_F`	エラー	L・F の直前
`_100`	エラー	数ではなく**変数名**と読まれ、cannot find symbol

置けるのは「数字と数字の間」だけ（Claude が作成）



値は変わらない。1_000_000 は 1000000。読みやすくするためだけの記号。

エラーの表示：置き場所がまちがっていると `illegal underscore`。

出典：JLS 3.10.1（Underscores are allowed as separators between digits that denote the integer／underscores may not appear immediately after 0x or 0b, or after the last digit in the numeral／Underscores may appear after the initial 0 in an octal numeral）／結果は JDK 17 で確かめたもの（1_.5・10_L・1.5_F も illegal underscore）

識別子・予約語・文脈キーワード

名前	結果	理由
`$a・_b・c3・名前`	OK	文字・数字・$・_。日本語も文字
`3c`	エラー	先頭に数字は不可
`my-var`	エラー	- は引き算の記号
`_`	エラー	Java 9 から予約語
`goto・const`	エラー	使っていないが予約語
`null・true`	エラー	リテラル
`int var = 3;`	OK	var は文脈キーワード
`class var {}`	エラー	型の名前には使えない
`int Main, String;`	OK	クラス名と同じ名前の変数も可（読みにくい）

予約語（51 個）：int・class・if など、名前にできない語。**goto と const** は使っていないのに予約されている。_ 1 文字も予約語。

文脈キーワード：var・record・sealed・permits・yield などは、決まった場所でだけキーワード。**変数名には使える**が、**型（クラス）の名前には使えない**。

true・false・null はキーワードではなく**リテラル**。それでも名前にはできない。

大文字と小文字は別：`count` と `Count` は別の名前。

出典：JLS 3.8 Identifiers（the first of which must be a Java letter／the ASCII dollar sign ($) and underscore (_)／TypeIdentifier: Identifier but not permits, record, sealed, var, or yield）／3.9 Keywords（51 character sequences … are reserved／The keywords const and goto are reserved／true and false are not keywords, but rather boolean literals）／結果は JDK 17 で確かめたもの

第3章

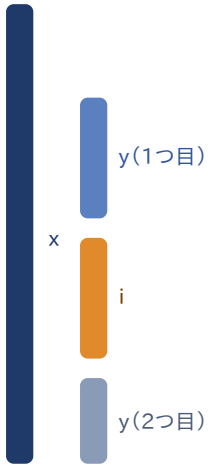
宣言・初期化・スコープ

ローカル変数の命は、宣言した所からブロックの終わりまで。

スコープはブロックで決まる

変数が使える範囲（Claude が作成）

```
void main(…) {
    int x = 10;
    if (x > 5) {
        int y = x * 2;
        …
    }
    for (int i = 0; …) {
        …
    }
    int y = 1; // OK
}
```



変数	スコープ
ブロックの変数	宣言した所から**そのブロックの } まで**
for の ( ) の変数	その for 文の中だけ。ループのあとは cannot find symbol
拡張 for の変数	ループの本体だけ
メソッドの引数	メソッドの中全体

ブロックが終われば名前は空く。if の中の y が終わったあとで、もう一度 `int y` を宣言してよい。2 つの for 文に同じ `int i` を書けるのも同じ理由。

1 つの宣言文の中:`int a = 1, b = a + 1;` のように、左の変数は右の初期化で使える。

出典：JLS 6.3 Scope of a Declaration（The scope of a local variable declared in a block … is the rest of the block, starting with the declaration's own initializer and including any further declarators to the right／The scope of a local variable declared in the ForInit part of a basic for statement includes … The contained Statement）

同じ名前を宣言できるとき・できないとき

NG:スコープの中でもう一度宣言

```
int x = 10;
{
  int x = 20;
}
```

```
error: variable x is already defined in method main(String[])
```

OK:ブロックが終わったあとで宣言

```
{
  int y = 20;
}
int y = 1;
```

OK:フィールドと同じ名前のローカル変数

```
static int x = 5;    // フィールド
… main の中 …
int x = 10;          // フィールドを隠す
System.out.println(x + Main.x);    // 15
```

組み合わせ	結果
ローカル変数 → 内側のブロックで同じ名前	エラー
内側のブロックの変数 → 外で後から同じ名前	OK
メソッドの引数と同じ名前のローカル変数	エラー
フィールドと同じ名前のローカル変数	OK(隠す)

隠す(シャドウイング)が許されるのは、種類がちがうとき。ローカル変数どうし(引数を含む)はエラー。フィールドは `Main.x`(static)や `this.x` で読める(#08・#09)。

出典:JLS 6.4 Shadowing and Obscuring(It is a compile-time error if the name of a local variable v is used to declare a new variable within the scope of v)／6.4.1 Shadowing／結果はJDK 17 で確かめたもの

宣言文の書き方と final

書き方	結果（JDK 17）
`int a = 1, b = a + 1, c;`	OK。b は 2
`int a, b = 3;` → a を表示	a might not have been initialized
`int a = 1, int b = 2;`	<identifier> expected
`int y = x = 5;`	OK。x も y も 5
`a = b = c = 7;`	OK。右から順に代入

`int a, b = 3;` の = 3 は b だけにかかる。a には何も入らない。

final のローカル変数	結果
`final int a = 1;`	OK
`final int a; a = 1;`	OK（あとで 1 回）
`final int a = 1; a = 2;`	cannot assign a value to final variable a
`final int a; a = 1; a = 2;`	might already have been assigned

final は「1 回だけ代入できる」。宣言のときに値がなくてもよい。final の付いた定数で初期化した int は「定数」として、byte への代入などで特別扱いされる（第4章）。

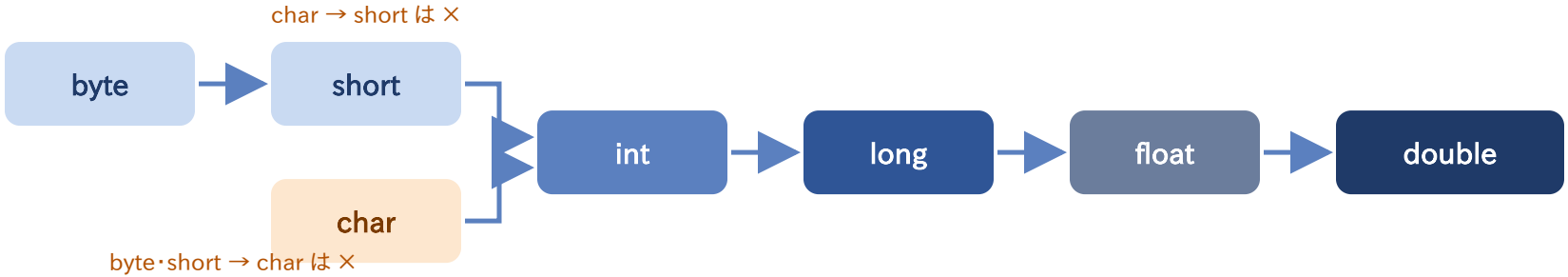
出典：JLS 14.4 Local Variable Declarations（VariableDeclaratorList: VariableDeclarator {, VariableDeclarator}）／14.4.2（the declarators are processed in order from left to right）／JLS 4.12.4 final Variables／結果は JDK 17 で確かめたもの

第4章

型変換とラッパークラス

広げるのは自動、縮めるのはキャスト。例外は「定数」と「+=」。

拡大変換 ― 自動で広がる向き



代入（キャストなし）	結果
`long l = i;`（i は int）	OK
`float f = l;`（l は long）	OK（精度が落ちることあり）
`int n = 'A';`	OK → 65
`char c = b;`（b は byte の変数）	lossy conversion from byte to char
`char c = s;`（s は short の変数）	lossy conversion from short to char

矢印の向きなら、代入するだけで変わる。float は 32 ビットだが、long（64 ビット）からも矢印がある（範囲が広いので）。ただし int・long → float、long → double は**下の桁が丸められる**ことがある。例外にはならない。

出典：JLS 5.1.2 Widening Primitive Conversion（byte to short, int, long, float, or double／short to int, long, float, or double／char to int, long, float, or double／int to long, float, or double／long to float or double／float to double／may result in loss of precision／never results in a run-time exception）／結果は JDK 17 で確かめたもの

縮小変換とキャスト

(byte) 200 が -56 になるわけ（Claude が作成）

int の 200（32 ビット）

0000 … 0000 0000	1100 1000

上位 24 ビットを捨てる

byte では先頭 1 ＝ 負 → 200−256 ＝ -56　1100 1000

```
System.out.println((byte) 200 + " " + (int) 3.99
                   + " " + (int) -3.99);
```

```
-56 3 -3
```

縮める変換	動き
整数 → 小さい整数	下位のビットだけ残す（符号が変わることも）
小数 → 整数	小数部分を捨てる（**0 の方向へ**）。3.99 → 3、-3.99 → -3。四捨五入ではない
double → float	近い float に丸める

キャストなしの縮小はコンパイルエラー（possible lossy conversion）。キャストを書けばコンパイルは通り、範囲外の値は**黙って変わる**（例外にはならない）。

特別ルール：`byte b = 10;` のように、**範囲内の int の定数**は byte・short・char にキャストなしで入る。`final int i = 10; byte b = i;` も OK。`int i = 10; byte b = i;` は NG。

出典：JLS 5.1.3 Narrowing Primitive Conversion（simply discards all but the n lowest order bits／the floating-point value is rounded to an integer value V using the round toward zero rounding policy）／5.2 Assignment Contexts（if the expression is a constant expression of type byte, short, char, or int: A narrowing primitive conversion may be used … byte theAnswer = 42; is allowed）／結果は JDK 17 で確かめたもの

計算の型と複合代入

計算の前に型をそろえる（数値の昇格）	
式に含まれる型	計算する型
double があれば	double
なければ float があれば	float
なければ long があれば	long
それ以外（byte・short・char・int）	int

byte + byte も int。char + char も int（'A' + 'B' は 131）。だから `b = b + 1;`（b は byte）は int を byte に入れることになり、エラー。

```
byte b = 10;
b = b + 1;        // lossy conversion from int to byte
b += 1;           // OK：b = (byte)(b + 1) と同じ
b++;              // OK
System.out.println(b);    // 12
byte c = 127;
c += 1;
System.out.println(c);    // -128（回りこむ）
```

複合代入（+= -= *= /= など）は、左辺の型への**キャストが自動で入る。**`E1 op= E2` は `E1 = (T)((E1) op (E2))`。

整数どうしの割り算は整数（5 / 2 は 2）。くわしくは #05 演算子。

出典：JLS 5.6 Numeric Contexts（If any expression is of type double, then the promoted type is double … In a numeric arithmetic context … the promoted type is int）／JLS 15.26.2 Compound Assignment Operators（E1 op= E2 is equivalent to E1 = (T) ((E1) op (E2)), where T is the type of E1）／結果は JDK 17 で確かめたもの

ラッパークラスとボクシング

基本データ型	ラッパークラス
int	Integer（名前がちがう）
char	Character（名前がちがう）
ほかの 6 つ	頭を大文字に：Byte・Short・Long・Float・Double・Boolean

ボクシング：基本データ型 → 対応するラッパー（自動）。**アンボクシング**：その逆。中身が **null** だと **NullPointerException**。

文字列から数へ：`Integer.parseInt("30")` は int、`Integer.valueOf("20")` は Integer。読めない文字列は NumberFormatException。

「広げてから包む」はできない。int の 10 は Integer にしか包めない。Long にしたいなら `10L`。

代入	結果（JDK 17）
`Integer a = 10;`	OK（ボクシング）
`int b = a;`	OK（アンボクシング）
`long g = Integer.valueOf(5);`	OK（取り出して広げる）
`Long f = 10;`	int cannot be … Long
`Double h = 1;`	int cannot be … Double
`Integer x = 10L;`	long cannot be … Integer
`Short s = 10;` `Character c = 65;`	OK（範囲内の int の定数）
`Integer x = null; int y = x;`	実行時 NullPointerException

出典：JLS 5.1.7 Boxing Conversion（From type int to type Integer）／5.1.8（If r is null, unboxing conversion throws a NullPointerException）／5.2（an unboxing conversion followed by a widening primitive conversion／A narrowing primitive conversion followed by a boxing conversion may be used if the variable is of type Byte, Short, or Character）／Java SE 17 API Integer（parseInt … NumberFormatException - if the String does not contain a parsable int）

第5章

var（ローカル変数型推論）

型を書かないだけで、型はある。初期化の式から 1 つに決まる。

var の決まり

書き方	結果（JDK 17）
`var a = 1;`	int（Integer ではない）
`var x = (String) null;`	String
`for (var i = 0; …)`・`for (var s : args)`	OK
`var a;`・`var a = null;`	cannot infer type
`var a = {1, 2};`・`var f = () -> "hello";`	cannot infer type
`var a = 1, b = 2;`	not allowed in a compound declaration
`var a[] = new int[2];`	not allowed as an element type of an array
フィールド・引数・戻り値の型	'var' is not allowed here

型はあとから変わらない

```
var a = 1;          // int に決まる
a = 'A';            // char → int は OK。a は 65
a = 2.5;            // lossy conversion from double to int
var z = 'a' + 1;    // int の 98
var var = "var";    // OK（var は変数名に使える）
```

使える場所：ローカル変数だけ（for・拡張 for・try-with-resources を含む）。**初期化が 1 つ**あり、その式だけで型が決まることが条件。

動的な型ではない：コンパイルのときに型が決まり、型を書いたのと同じになる。JavaScript の var とはちがう。

出典：JLS 14.4（It is a compile-time error if the LocalVariableType is var and any of the following are true: More than one VariableDeclarator is listed／The VariableDeclaratorId has one or more bracket pairs／The VariableDeclarator lacks an initializer／The initializer … is an ArrayInitializer）／14.4.1（It is a compile-time error if T is the null type／an error occurs if it is a lambda expression）／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
何が表示されるか（初期値）	フィールド・配列は 0・false・null・0.0。**ローカル変数は値なし**でコンパイルエラー
コンパイルできる代入は	拡大はそのまま、縮小はキャスト。**範囲内の int の定数**は byte・short・char に入る。byte → char は不可
リテラルの型	整数は int（L で long）、小数は double（F で float）。0x・0・0b の値
アンダースコア	数字と数字の間だけ。先頭の _ は変数名になる
変数名として正しいのは	先頭に数字なし、$ と _ と日本語は可、_ 1 文字・goto・null は不可、var は可
スコープ	ブロックの } まで。for の i はループの外で使えない。ローカル変数どうしの重複はエラー
計算結果	byte・short・char は int で計算。int のまま回りこむ。+= は自動キャスト
var	初期化 1 つ・null 不可・{ } 不可・ローカルだけ。型は int など基本データ型のまま

出典：このPDFの各ページの資料

まとめ

基本データ型
byte 8・short 16・char 16（符号なし）・int 32・long 64・float 32・double 64・boolean

リテラルと名前
L・F・0x・0・0b・_、先頭に数字なし、予約語と文脈キーワード

宣言とスコープ
ローカル変数は初期値なし。ブロックの終わりまで。重複はエラー

型変換・ラッパー・var
広げるのは自動、縮めるのはキャスト。int ⇔ Integer。var は 1 つの型に決まる

エラーの表示	原因
might not have been initialized	ローカル変数に値がない
possible lossy conversion	キャストなしの縮小変換
integer number too large	int の範囲をこえるリテラル（L がない）
illegal underscore	_ の置き場所
is already defined	スコープの中で同じ名前を宣言
cannot infer type	var の型が決まらない

次は #03 文字列。String の不変性・メソッド・StringBuilder・テキストブロックへ進む。

出典：このPDFの各ページの資料